

性能优化-大地图

机器：IQOO neo8 ， 23年 2000元档位

场景资源来源: AssetStore商店

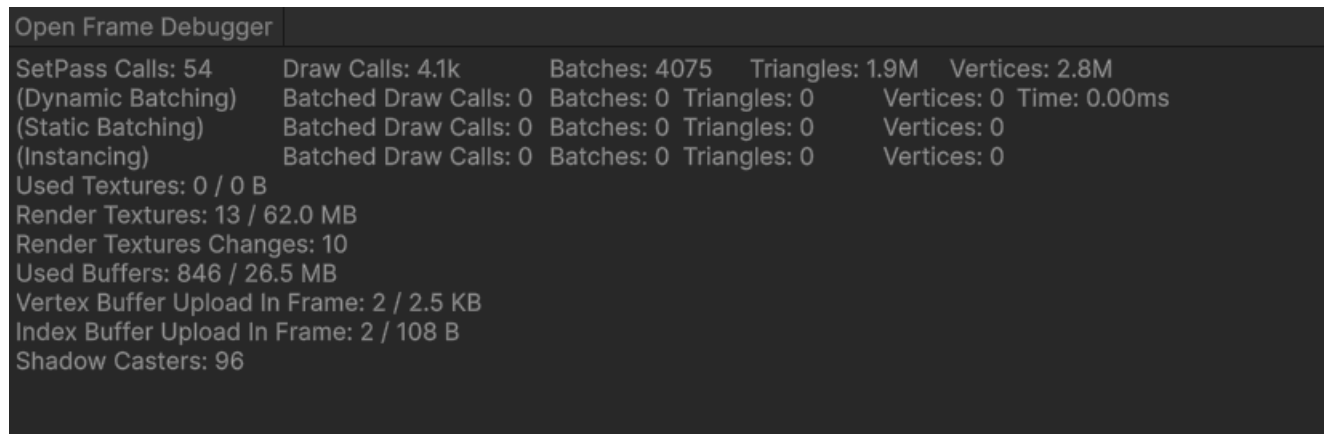
<https://assetstore.unity.com/packages/3d/environments/pure-nature-2-meadows-269085>

打包到手机上测试

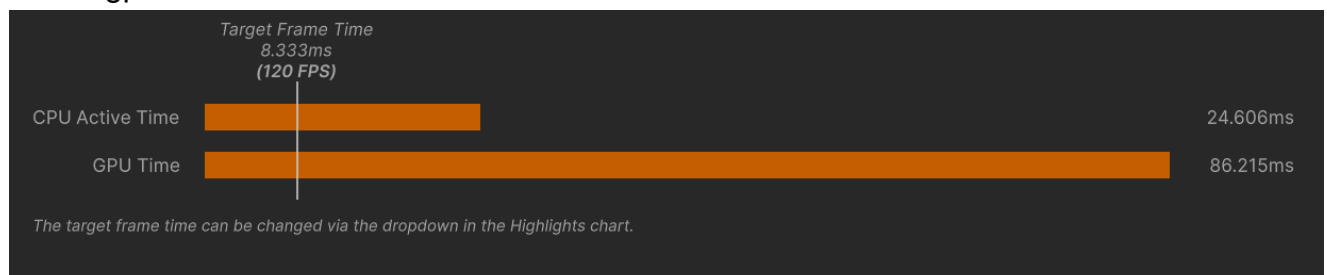
不降频的时候19帧，降频后11帧（手机发热后会降频）



profile 统计面板



瓶颈在gpu



1) SetPass Calls很低，只有54，说明SRPBatch生效了，场景里的草和树的shader变体并不多，因此可以大量进行合批。

2) dc 看起来很高 4.1k

dc来源：树（没有instancing，一颗树算一个，大概2k+） + 草（有instancing） + 阴影 + 其

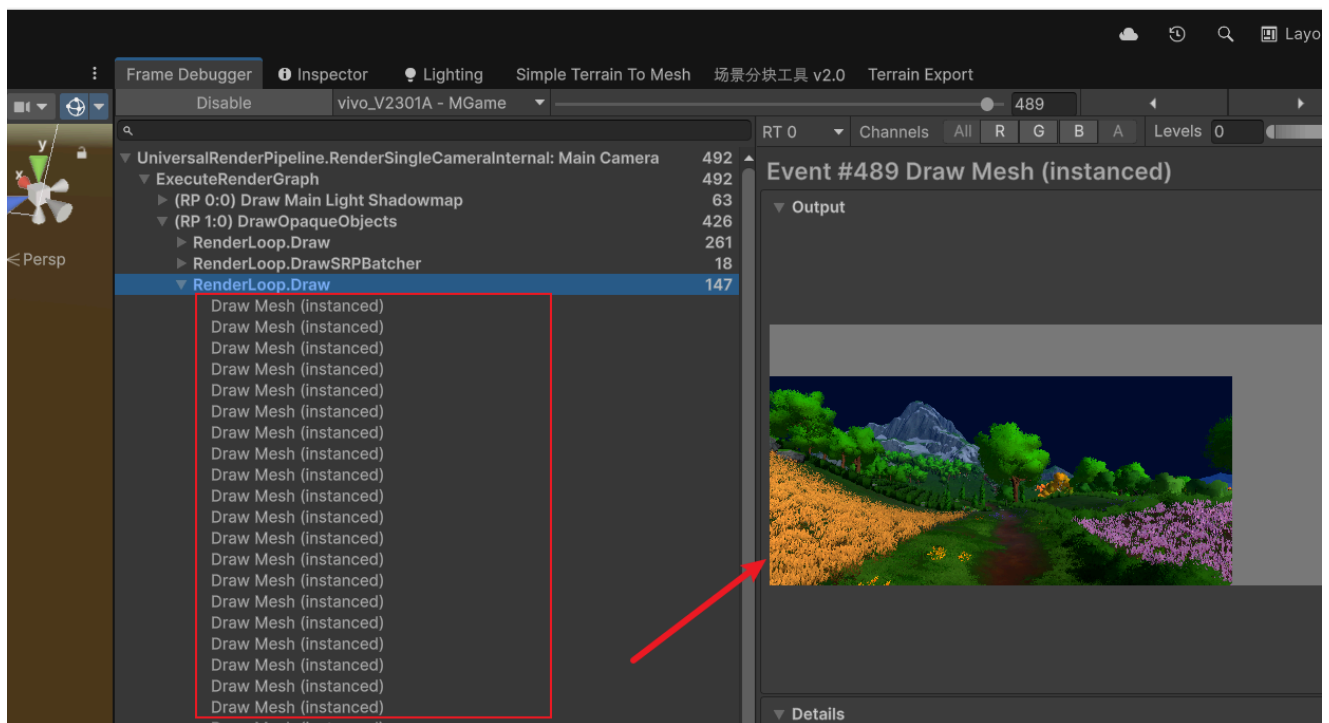
他

dc数量多，需要提交的网格数据和材质属性多，都会导致cpu在多线程的submit耗时比较高。

Hierarchy	Live	Main Thread	CPU:91.61ms	GPU:--ms		
Overview	Total	Self	Calls	OC Alloc	Time ms	Self ms
▼ PlayerLoop	99.9%	0.2%	1	32 B	91.55	0.19
▼ PostLateUpdate.FinishFrameRendering	97.3%	0.1%	1	0 B	89.15	0.14
▶ Gfx.WaitForPresentOnGfxThread	69.0%	0.0%	1	0 B	63.24	0.00
▼ RenderPipelineManager.DoRenderLoop_Internal() [Invoke]	27.8%	0.1%	1	0 B	25.50	0.15
▼ InL_UniversalRenderTotal	27.6%	0.1%	1	0 B	25.34	0.13
▼ InL_RenderCameraStack	27.4%	0.4%	2	0 B	25.16	0.43
▶ InL_UniversalRenderPipeline.RenderSingleCameraInternal: Main Camera	13.1%	0.5%	1	0 B	12.07	0.51
▶ InL_ScriptableRenderContext.Submit	10.4%	0.2%	2	0 B	9.53	0.19
▶ InL_UniversalRenderPipeline.RenderSingleCameraInternal: UI Camera	2.8%	0.3%	1	0 B	2.64	0.32
▶ InL_UniversalRenderPipeline.CreateCameraData	0.2%	0.1%	2	0 B	0.25	0.12
▶ InL_UniversalRenderPipeline.InitializeAdditionalCameraData	0.1%	0.1%	3	0 B	0.14	0.14
▶ InL_UpdateVolumeFramework	0.0%	0.0%	2	0 B	0.06	0.03
▶ InL_RenderPipeline.EndCameraRendering	0.0%	0.0%	2	0 B	0.00	0.00
▶ InL_RenderPipeline.BeginCameraRendering	0.0%	0.0%	2	0 B	0.00	0.00

3) Batches, 这里看Batches的数量以及下方统计，可以看出动态批处理，静态批处理，Instancing都没有生效，合批数据都是0，且没有合并dc，基本一个dc就一个batch。

但是看FrameDebugger，可以看出是统计错误，实际上instancing是生效了，下图可以看到所有的树、草都是通过gpu instance实例化的，开启方式是材质默认勾选了 enabled instancing。



从renderdoc也可以看出，每次绘制都是几百个物体，因此确定是有gpu instance 优化的。

579-9411	▼ DrawOpaqueObjects	8197.55188
580-1471	▶ RenderLoop.Draw	566.04165
1473-2957	▶ RenderLoop.DrawSRPBatcher	1538.0208
2959-5048	▶ RenderLoop.DrawSRPBatcher	1548.9583
5050-7066	▶ RenderLoop.DrawSRPBatcher	2140.1562
7068-9069	▶ RenderLoop.DrawSRPBatcher	1762.18746
9071-9409	▼ RenderLoop.Draw	642.18748
9072	vkCmdBindPipeline(Graphics Pipeline 12154)	
9073	vkCmdBindDescriptorSets(0, { Descriptor Set 11850, Descriptor Set 13159 })	
9074	vkCmdBindVertexBuffers(0, { Lupin1 })	
9075	vkCmdBindIndexBuffer(Lupin1, VK_INDEX_TYPE_UINT16)	
9076	vkCmdDrawIndexed(48, 454)	1.14583
9077	vkCmdBindDescriptorSets(0, { Descriptor Set 11850, Descriptor Set 13159 })	
9078	vkCmdDrawIndexed(48, 454)	1.71875
9079	vkCmdBindDescriptorSets(0, { Descriptor Set 11850, Descriptor Set 13159 })	
9080	vkCmdDrawIndexed(48, 454)	23.125
9081	vkCmdBindDescriptorSets(0, { Descriptor Set 11850, Descriptor Set 13159 })	
9082	vkCmdDrawIndexed(48, 454)	18.64583
9083	vkCmdBindDescriptorSets(0, { Descriptor Set 11850, Descriptor Set 15272 })	
9084	vkCmdDrawIndexed(48, 227)	1.19792
9085	vkCmdBindDescriptorSets(0, { Descriptor Set 11850, Descriptor Set 12183 })	
9086	vkCmdDrawIndexed(48, 363)	100.57291
9087	vkCmdBindDescriptorSets(0, { Descriptor Set 11850, Descriptor Set 12891 })	
9088	vkCmdDrawIndexed(48, 454)	1.66667
9089	vkCmdBindDescriptorSets(0, { Descriptor Set 11850, Descriptor Set 12891 })	
9090	vkCmdDrawIndexed(48, 454)	1.45833

4) Triangles 190w , Vertices 280w

其中地形 12w, 山 16w, 其他是草和树160w

虽然terrain有做lod, 视野范围外的物体也有剔除（依赖terrain的分patch渲染），但是整体模型的面数偏高，在移动端建议面数是在50w以下。

RenderDoc 命令执行时间是12ms，注意这里不包括带宽、热降频等产生的影响，只包括执行命令的时间。从编辑器看gpu的总耗时是86ms，对应11帧。

Duration (µs)
12036.51012

gpu执行的命令（API Calls）一共5414，显然有点多

左下角那个9600是Event，包含了虚拟事件（方便我们调试）和API Call。

Texture Viewer x
Statistics x

Stats for com.unity.mgame_2025.12.07_22.24_frame622.rdc.
File size: 246.18MB (433.95MB uncompressed, compression ratio 1.76:1)
Persistent Data (approx): 32.28MB, Frame-initial data (approx): 399.53MB
*** Summary ***
Draw calls: 4075
Dispatch calls: 0
API calls: 5414
API:Draw/Dispatch call ratio: 1.32859
165 Textures - 291.06 MB (291.04 MB over 32x32), 11 RTs - 108.59 MB.
Avg. tex dimension: 1512.28x1684.53 (1582.59x1763.7 over 32x32)
645 Buffers - 41.39 MB total 1.72 MB IBs 9.75 MB VBs.
441.04 MB - Grand total GPU buffer + texture load.

94	vkCmdPipelineBarrier({ URP Z-Bin Buffer })
95	vkCmdCopyBuffer(Scratchbuffer page, URP Z-Bin Buffer)
96	vkCmdPipelineBarrier({ URP Tile Buffer })
97	vkCmdCopyBuffer(Scratchbuffer page, URP Tile Buffer)
98	=> vkQueueSubmit(15)[0]: vkEndCommandBuffer(Baked Command Buffer 15)
99	=> vkQueueSubmit(15)[1]: vkBeginCommandBuffer(Baked Command Buffer 15)
100	vkCmdResetQueryPool(Query Pool 168)
101	> WaitForRenderJobs
103	> CustomRenderTextures.Update
105	> ReflectionProbes.Update
107	vkCmdWriteTimestamp(Query Pool 168, 5814)
108-9600	> FrameTime.GPU
9602	vkCmdPipelineBarrier({ Swapchain Image 435 })
9603	=> vkQueueSubmit(15)[14]: vkEndCommandBuffer(Baked Command Buffer 15)
9604	vkGetFenceStatus(Fence 15843)
9605	vkQueuePresentKHR(Swapchain Image 435)

主要的耗时在阴影和绘制不透明物体

136	=> vkQueueSubmit(15)[1]: vkEndCommandBuffer(Baked Command Buffer 15823)	
137	=> vkQueueSubmit(15)[2]: vkBeginCommandBuffer(Baked Command Buffer 15830)	
138	vkCmdBeginRenderPass2(D=Clear)	
139-565	Draw Main Light Shadowmap	2293.07286
567	Setup Camera Properties	0.00
569	vkCmdEndRenderPass2(D=Store)	
570	vkCmdPipelineBarrier({ _CameraTargetAttachmentA_2240x1008_B10G11R11_UFloatPack32_Tex2D })	
571	vkCmdPipelineBarrier({ _CameraDepthAttachment_2240x1008_D24_UNorm_S8_UInt_Tex2D })	
572	=> vkQueueSubmit(15)[2]: vkEndCommandBuffer(Baked Command Buffer 15830)	
573	=> vkQueueSubmit(15)[3]: vkBeginCommandBuffer(Baked Command Buffer 15832)	
574	vkCmdPipelineBarrier({ _MainLightShadowmapTexture_1024x1024_Shadowmap })	
575	=> vkQueueSubmit(15)[3]: vkEndCommandBuffer(Baked Command Buffer 15832)	
576	=> vkQueueSubmit(15)[4]: vkBeginCommandBuffer(Baked Command Buffer 15831)	
577	vkCmdBeginRenderPass2(C=Clear, DS=Clear)	
578	vkCmdWriteTimestamp(Query Pool 168, 5821)	
579-9411	DrawOpaqueObjects	8197.55188
580-1471	RenderLoop.Draw	566.04165
581	vkCmdSetScissor(0, { { 0, 0 }, { 2240, 1008 } })	
582	vkCmdSetViewport(0, { { 0.00, 0.00, 2240.00, 1008.00 } })	

移动平台只开了一级的csm，会对场景里所有在阴影距离内的物体都渲染一遍

ShadowCaster pass，这个数量算正常，阴影距离只有50米，因此绘制数量不多。

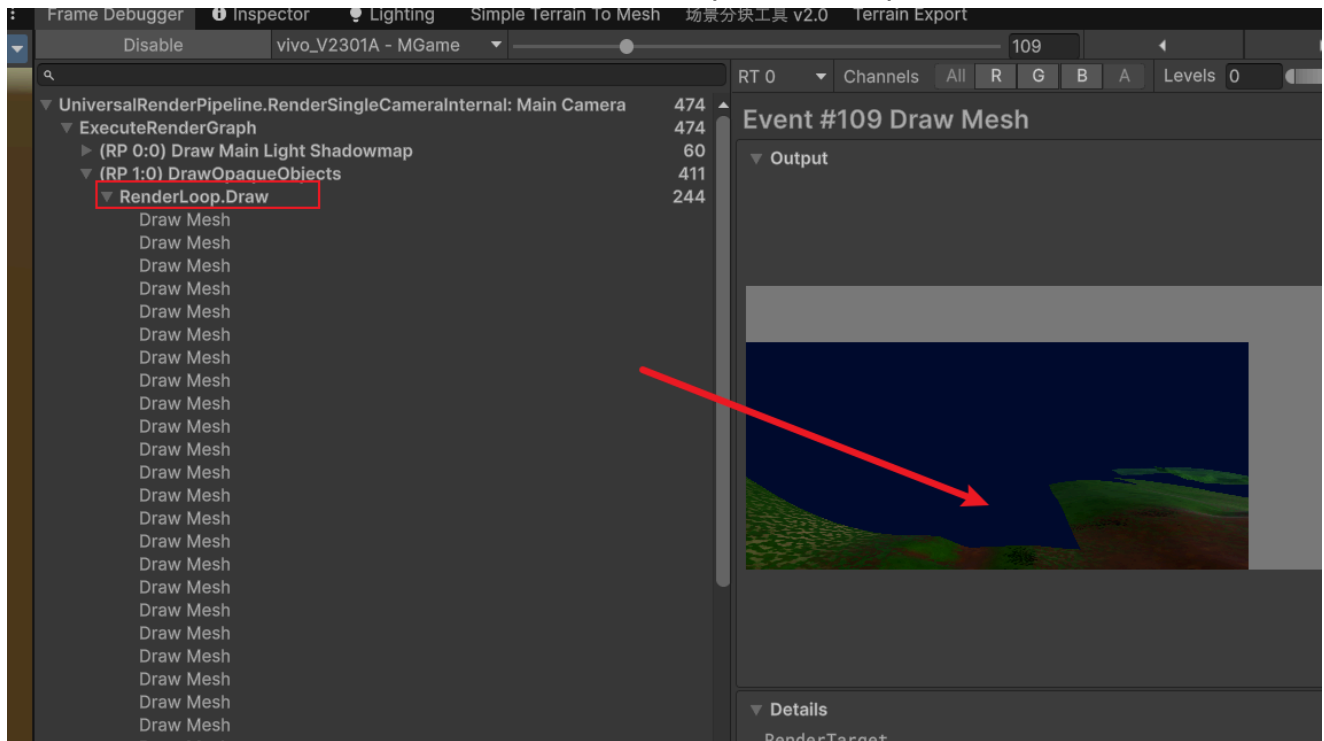
UniversalRenderPipeline.RenderSingleCameraInternal: Main Camera	492	Event #63 Draw Mesh	
ExecuteRenderGraph	492	Output	
(RP 0:0) Draw Main Light Shadowmap	63		
MainLightShadow	63		
Shadows.Draw	42		
Shadows.DrawSRPBatcher	5		
Shadows.Draw	16		
(RP 1:0) DrawOpaqueObjects	426		
RenderLoop.Draw	261		
RenderLoop.DrawSRPBatcher	18		
RenderLoop.Draw	147		
(RP 2:0) Copy Depth for HZB	1		
Draw Procedural			
(RP 3:0) DrawSkybox	1		
Camera.RenderSkybox	1		
Draw Mesh			
(RP 4:0) CopyDepth	1		
CopyDepth	1		
Draw Procedural			
UniversalRenderPipeline.RenderSingleCameraInternal: UI Camera	8		
ExecuteRenderGraph	8		
(RP 6:0) DrawTransparentObjects	1		
RenderLoop.Draw	1		
Canvas.RenderSubBatch	1		
Draw Mesh			
(RP 7:0) BlitFinalToBackBuffer	1		
Draw Procedural			
(RP 7:0) DrawScreenSpaceUI	6		
UI.RenderOverlays	6		
Clear (depth+stencil)			
Canvas.RenderOverlays	5		
Draw Mesh			
Draw Mesh			
Draw Mesh			
Draw Mesh			
Draw Mesh			
GUITexture.Draw	1		
Draw Dynamic			

不透明物体的绘制主要分3块，地形，

(RP 1:0) DrawOpaqueObjects	426
RenderLoop.Draw	261
RenderLoop.DrawSRPBatcher	18
RenderLoop.Draw	147

1) 第一块RenderLoop.Draw 是地形的渲染， terrain是分patch渲染的，近距离高分辨率网

格，远距离简化网格。而且只会渲染视野范围内的patch，每个patch再单独绘制草和树。



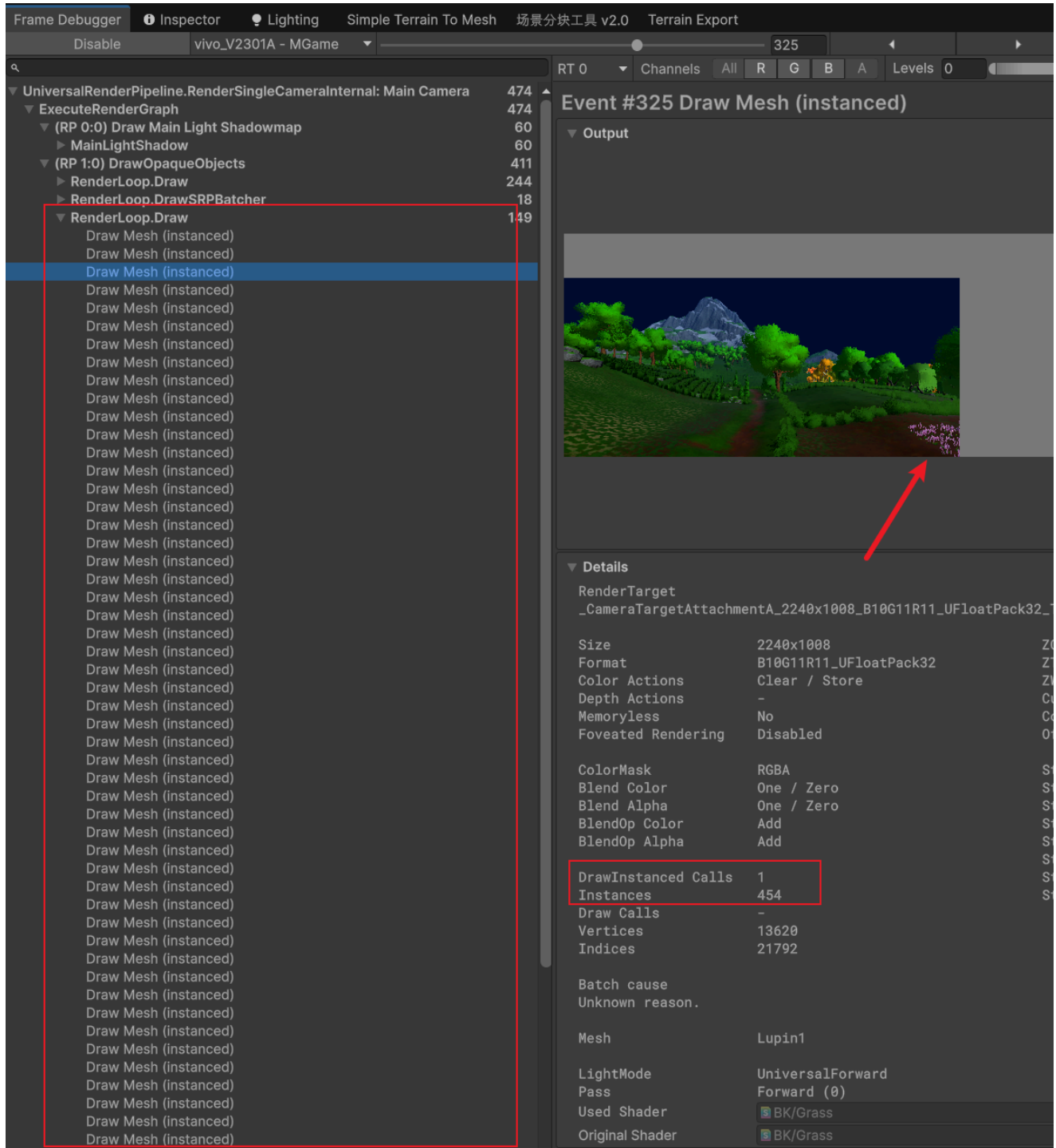
2) 第二块是RenderLoop.DrawSRPBatcher，树的材质勾选了 enabled instancing 但是没有生效，推断是terrain有自己的渲染策略。Terrain setting 中 Tree Details标签下也没有相关的实例化选项。

The screenshot displays the Unity Hierarchy and Hierarchy views. The Hierarchy view on the left shows the rendering pipeline for a main camera, with the 'RenderLoop.DrawSRPBatcher' node highlighted. The Hierarchy view on the right shows the 'Event #313 SRP Batch' node, which is also highlighted. The 'Details' panel for the selected batch shows the following statistics:

Property	Value	Unit
DrawInstanced Calls	-	
Instances	-	
Draw Calls	11	
Vertices	35454	
Indices	75876	

3) 第三块是草的实例化，是用了gpu instancing 的
可以看到并没有一次性绘制所有的粉色的花，是因为terrain分patch渲染。

DrawMeshInstance 每次实例化数量的上限是1023个，下面看到基本每次都在几百个左右。

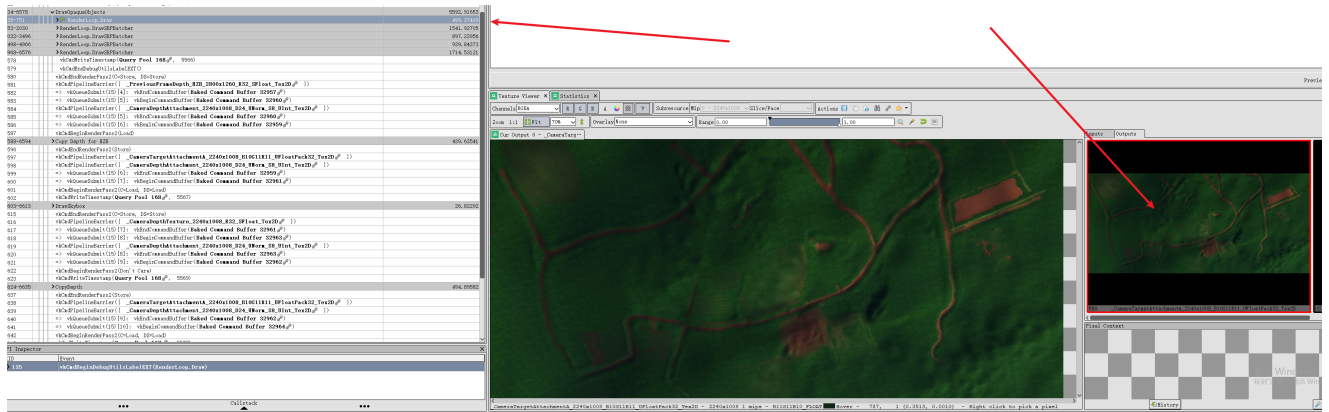


地形消耗

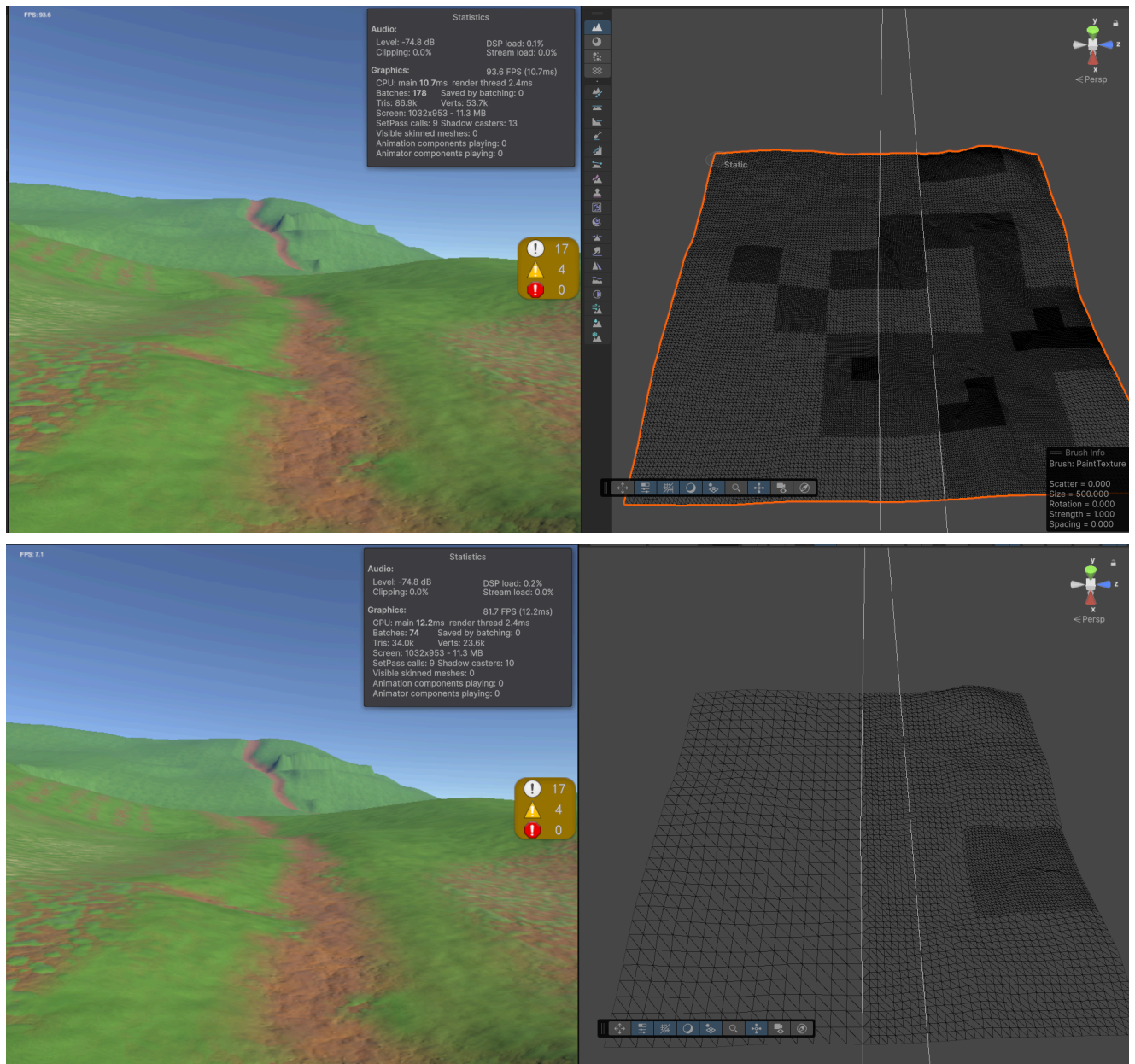
1) 从renderdoc来看，耗时在499us

因为做了分patch，只渲染视野范围内，渲染性能其实还行
渲染过程是通过采样权重图，融合各个layer的纹理。

性能优化-大数据



2) 通过调整pixel error，让远处使用更低分辨率的网格，像这个视角
pixel error 填1, 8.5w面
pixel error 填10, 3.4w面



3) 因此unity的terrain本身其实做了很多优化，在这里并不算瓶颈。只是如果在移动平台，要适配低端机，那么就需要更加高自由度的调整性能参数。

草和树的消耗

这些加起来一共是 7ms，这个数据太高了（面数高，draw命令多，shader复杂度高）

>RenderLoop.DrawSRPBatcher	1554.32288
>RenderLoop.DrawSRPBatcher	1649.79163
>RenderLoop.DrawSRPBatcher	1776.14579
>RenderLoop.DrawSRPBatcher	2153.2812
>RenderLoop.Draw	202.08333

分析帧数低的原因：

- 1) 带宽瓶颈，190w面数太高，应该控制到50w以下
- 2) cpu的submit耗时太高（取决于dc数量，cpu需要上传的网格数据和材质属性），不过这不是瓶颈，瓶颈在gpu。
- 3) GPU的API Call 太高，draw命令太多导致的
- 4) 使用terrain无法更好的控制渲染性能，只能设置各种选项，自由度不足。
- 5) terrain的性能本身已经还可以，但在低端机上还不够低

当前比较好的地方

- 1) terrain分patch渲染，视野范围外的patch都剔除了，遮挡比较多的角度下帧率会变高。
- 2) 草做了gpu instance
- 3) 树做了4级lod
- 4) 阴影距离50m是合适的，csm开启1级也是合适的

优化方向

- 1) 通过gpu instancing 实例化树和草，进行视锥剔除 + HZB剔除 + 距离剔除
- 2) 树的面数需要降低，草面数本身很低了不需要处理。
- 3) 地形转网格，分块，烘焙，流式加载
- 4) 瓶颈在GPU，全面降低set pass call，dc，Triangles, Vertices，API Call 这些性能指标。
- 5) cpu、内存、显存方面无需处理，暂不是瓶颈

树种类：39种，

当前4级lod的面数：4000, 1400, 500, 100

优化：只要后面3级

草种类：29种

没有lod，面数10 - 90 都有

面数不高，暂不优化

优化后

启动游戏后，一开始是100帧，机器测试一段时间温度升高，cpu降频率了，会稳定在60帧左右



性能指标：

面板上的Triangles 只包括地面和山：3w

gpu instance的树：11w

gpu instance的草：8w

总面数: $3 + 11 + 8 = 22w$ (之前是190w)

SetPass Call 基本来自草和树

阴影：50米内，草和树都会投射阴影和接收阴影

Open Frame Debugger					
SetPass Calls: 286	Draw Calls: 318	Batches: 318	Triangles: 33.2k	Vertices: 22.7k	
(Dynamic Batching)	Batched Draw Calls: 0	Batches: 0	Triangles: 0	Vertices: 0	Time: 0.00ms
(Static Batching)	Batched Draw Calls: 0	Batches: 0	Triangles: 0	Vertices: 0	
(Instancing)	Batched Draw Calls: 0	Batches: 0	Triangles: 0	Vertices: 0	
Used Textures: 0 / 0 B					
Render Textures: 9 / 49.9 MB					
Render Textures Changes: 10					
Used Buffers: 873 / 198.5 MB					
Vertex Buffer Upload In Frame: 1 / 96 B					
Index Buffer Upload In Frame: 1 / 12 B					
Shadow Casters: 102					

hzb 并不是一定会有优化，取决于视角，甚至会带来负优化。因此关闭了。

树的数量 3468颗

Debug	
Enable Debug	☒
Readback Interval (s)	1
Culling Statistics:	
Total Instances	9188
After Frustum Culling	4197
After LOD Culling	3468
After HZB Culling (Final)	3468
Rendering Statistics:	
Final Triangles	116730
Final Vertices	177924

草的数量 3020颗

Culling Statistics:	
Total Instances	1685121
After Frustum Culling	3020
After HZB Culling (Final)	3020
Rendering Statistics:	
Final Triangles	82226
Final Vertices	130082

set pass call : 286 为啥这么多, 网格不同就需要分开调用drawMeshInstanceIndirect
39种树 x 3级lod x 2个submesh = 最大可能数是234

草 $29 \times 1 = 29$

这里要注意, 即使视野内没有任何内容, 也会发起这么多次的drawMeshInstanceIndirect
因为cpu并不知道剔除后数量是否为0, 要知道的话只能回读, 回读会导致cpu堵塞。
因此drawMesh 导致的 set pass call 是固定的, 但没有内容的话, 消耗会非常低
数量为0的时候, 消耗是0.64us, 忽略

Draw Calls: 318 (之前是4000)

Batches: 318

Triangles: 3.3w Vertices: 2.2w 这些是场景里的山和地形, 网格都精简过, (之前一共30w)

RenderDoc

GPU 执行指令总耗时 5ms (之前是12ms)

		
	Duration (μs)	
		5154.99987

视椎剔除的耗时 2ms

其中草的视锥剔除比较耗时， 一共168w颗草

原因是需要判断的草太多了，这里有两条特别久的，是因为这两种草的数量最多。

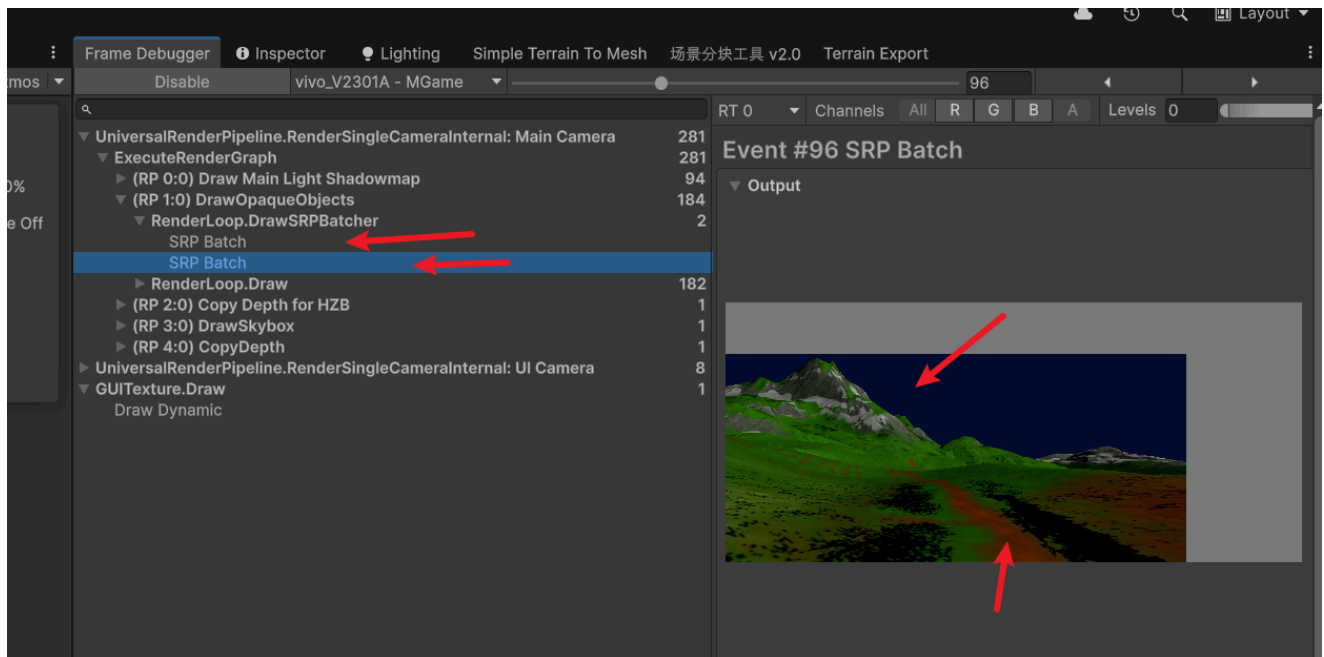
computer shader 中的剔除运算应该不是瓶颈，瓶颈应该是在于调用的量级（遇到了带宽瓶颈，而不是计算的瓶颈）。因为把相机丢很远，大部分草应该在廉价的距离剔除就返回了，但耗时基本没变。

158-164	GPU_FrustumCuller_Dispatch	1049.99997
171-177	GPU_FrustumCuller_Dispatch	1034.27081
184-190	GPU_FrustumCuller_Dispatch	160.67708
197-203	GPU_FrustumCuller_Dispatch	228.74999
210-216	GPU_FrustumCuller_Dispatch	125.46875
223-229	GPU_FrustumCuller_Dispatch	23.90625
236-242	GPU_FrustumCuller_Dispatch	6.09375
249-255	GPU_FrustumCuller_Dispatch	6.97917
262-268	GPU_FrustumCuller_Dispatch	5.625
275-281	GPU_FrustumCuller_Dispatch	25.67708
288-294	GPU_FrustumCuller_Dispatch	35.57292
301-307	GPU_FrustumCuller_Dispatch	18.95833
314-320	GPU_FrustumCuller_Dispatch	17.23958
327-333	GPU_FrustumCuller_Dispatch	10.3125
340-346	GPU_FrustumCuller_Dispatch	9.16667
353-359	GPU_FrustumCuller_Dispatch	10.9375
366-372	GPU_FrustumCuller_Dispatch	12.96875
379-385	GPU_FrustumCuller_Dispatch	7.65625
392-398	GPU_FrustumCuller_Dispatch	22.86458
405-411	GPU_FrustumCuller_Dispatch	8.75
418-424	GPU_FrustumCuller_Dispatch	38.75
431-437	GPU_FrustumCuller_Dispatch	6.51042
444-450	GPU_FrustumCuller_Dispatch	4.84375
457-463	GPU_FrustumCuller_Dispatch	6.5625
470-476	GPU_FrustumCuller_Dispatch	7.29167
483-489	GPU_FrustumCuller_Dispatch	7.29167
496-502	GPU_FrustumCuller_Dispatch	6.09375
509-515	GPU_FrustumCuller_Dispatch	5.57292
522-528	GPU_FrustumCuller_Dispatch	39.01042

其余耗时主要是两个相机的渲染

Controls			Filter	Settings & Help
TreeRendering_Optimized			UniversalRenderPipeline.RenderSingleCameraInternal: Main Camera	ExecuteRenderGraph
EID	Name	Duration (μs)		
520	vkCmdEndDebugUtilsLabelEXT()			
521-2247	ExecuteRenderGraph	1984.27078		
522	Initialize Frame	0.00		
524	Set Motion Vector Global Matrices	0.00		
526-534	Setup Forward Lights	0.00		
536	Setup Debug Properties	0.00		
538	Setup Camera Properties	0.00		
540	vkCmdPipelineBarrier(MainLightShadowmapTexture_1024x1024_Shadowmap)			
541	=> vkQueueSubmit(16) [1]: vkEndCommandBuffer(Baked Command Buffer 19389)			
542	=> vkQueueSubmit(16) [2]: vkBeginCommandBuffer(Baked Command Buffer 19400)			
543	vkCmdPipelineBarrier(Buffer 13215)			
544	vkCmdPipelineBarrier(VkBufferMemoryBarrier[29])			
545	vkCmdPipelineBarrier(VkBufferMemoryBarrier[30])			
546	=> vkQueueSubmit(16) [2]: vkEndCommandBuffer(Baked Command Buffer 19400)			
547	=> vkQueueSubmit(16) [3]: vkBeginCommandBuffer(Baked Command Buffer 19399)			
548	vkCmdBeginRenderPass2(D=Clear)			
549-1181	Draw Main Light Shadowmap	677.96873		
1183	Setup Camera Properties	0.00		
1185	vkCmdEndRenderPass2(D=Store)			
1186	vkCmdPipelineBarrier(CameraTargetAttachmentA_2240x1008_B10G11R11_UFloatPack32_Tex2D)			
1187	vkCmdPipelineBarrier(CameraDepthAttachment_2240x1008_D24_UNorm_S8_UInt_Tex2D)			
1188	=> vkQueueSubmit(16) [3]: vkEndCommandBuffer(Baked Command Buffer 19399)			
1189	=> vkQueueSubmit(16) [4]: vkBeginCommandBuffer(Baked Command Buffer 19402)			
1190	vkCmdPipelineBarrier(MainLightShadowmapTexture_1024x1024_Shadowmap)			
1191	=> vkQueueSubmit(16) [4]: vkEndCommandBuffer(Baked Command Buffer 19402)			
1192	=> vkQueueSubmit(16) [5]: vkBeginCommandBuffer(Baked Command Buffer 19401)			
1193	vkCmdBeginRenderPass2(C=Clear, DS=Clear)			
1194	vkCmdWriteTimestamp(Query Pool 168, 6165)			
1195-2178	DrawOpaqueObjects	350.67707		
2180	vkCmdEndRenderPass2(C=Store, DS=Store)			
2181	vkCmdPipelineBarrier(PreviousFrameDepth_HZB_2800x1260_R32_SFloat_Tex2D)			
2182	=> vkQueueSubmit(16) [5]: vkEndCommandBuffer(Baked Command Buffer 19401)			
2183	=> vkQueueSubmit(16) [6]: vkBeginCommandBuffer(Baked Command Buffer 19404)			
2184	vkCmdPipelineBarrier(CameraDepthAttachment_2240x1008_D24_UNorm_S8_UInt_Tex2D)			
2185	=> vkQueueSubmit(16) [6]: vkEndCommandBuffer(Baked Command Buffer 19404)			
2186	=> vkQueueSubmit(16) [7]: vkBeginCommandBuffer(Baked Command Buffer 19403)			
2187	vkCmdBeginRenderPass2(Load)			
2188-2194	Copy Depth for HZB	438.02082		
2196	vkCmdEndRenderPass2(Store)			
2197	vkCmdPipelineBarrier(CameraTargetAttachmentA_2240x1008_B10G11R11_UFloatPack32_Tex2D)			
2198	vkCmdPipelineBarrier(CameraDepthAttachment_2240x1008_D24_UNorm_S8_UInt_Tex2D)			
2199	=> vkQueueSubmit(16) [7]: vkEndCommandBuffer(Baked Command Buffer 19403)			
2200	=> vkQueueSubmit(16) [8]: vkBeginCommandBuffer(Baked Command Buffer 19405)			
2201	vkCmdBeginRenderPass2(C=Load, DS=Load)			
2202	vkCmdWriteTimestamp(Query Pool 168, 6167)			
2203-2213	DrawSkybox	63.90625		
2215	vkCmdEndRenderPass2(C=Store, DS=Store)			
2216	vkCmdPipelineBarrier(CameraDepthTexture_2240x1008_R32_SFloat_Tex2D)			
2217	=> vkQueueSubmit(16) [8]: vkEndCommandBuffer(Baked Command Buffer 19405)			
2218	=> vkQueueSubmit(16) [9]: vkBeginCommandBuffer(Baked Command Buffer 19407)			
2219	vkCmdPipelineBarrier(CameraDepthAttachment_2240x1008_D24_UNorm_S8_UInt_Tex2D)			
2220	=> vkQueueSubmit(16) [9]: vkEndCommandBuffer(Baked Command Buffer 19407)			
2221	=> vkQueueSubmit(16) [10]: vkBeginCommandBuffer(Baked Command Buffer 19406)			
2222	vkCmdBeginRenderPass2(Don't Care)			
2223	vkCmdWriteTimestamp(Query Pool 168, 6169)			
2224-2235	CopyDepth	453.69791		
2237	vkCmdEndRenderPass2(Store)			
2238	vkCmdPipelineBarrier(CameraTargetAttachmentA_2240x1008_B10G11R11_UFloatPack32_Tex2D)			
2239	vkCmdPipelineBarrier(CameraDepthAttachment_2240x1008_D24_UNorm_S8_UInt_Tex2D)			
2240	=> vkQueueSubmit(16) [10]: vkEndCommandBuffer(Baked Command Buffer 19406)			
2241	=> vkQueueSubmit(16) [11]: vkBeginCommandBuffer(Baked Command Buffer 19408)			
2242	vkCmdBeginRenderPass2(C=Load, DS=Load)			
2243	vkCmdWriteTimestamp(Query Pool 168, 6173)			
2244-2245	DrawTransparentObjects	0.00		
2247	vkCmdEndRenderPass2(C=Store, DS=Store)			
2248	vkCmdEndDebugUtilsLabelEXT()			
2249	vkCmdEndDebugUtilsLabelEXT()			
2250-2347	UniversalRenderPipeline.RenderSingleCameraInternal: UI Camera	411.51041		
2349-2362	GUITexture.Draw	6.19792		

两个批次分别是地面和山， shader变体比较统一，都有SRP优化

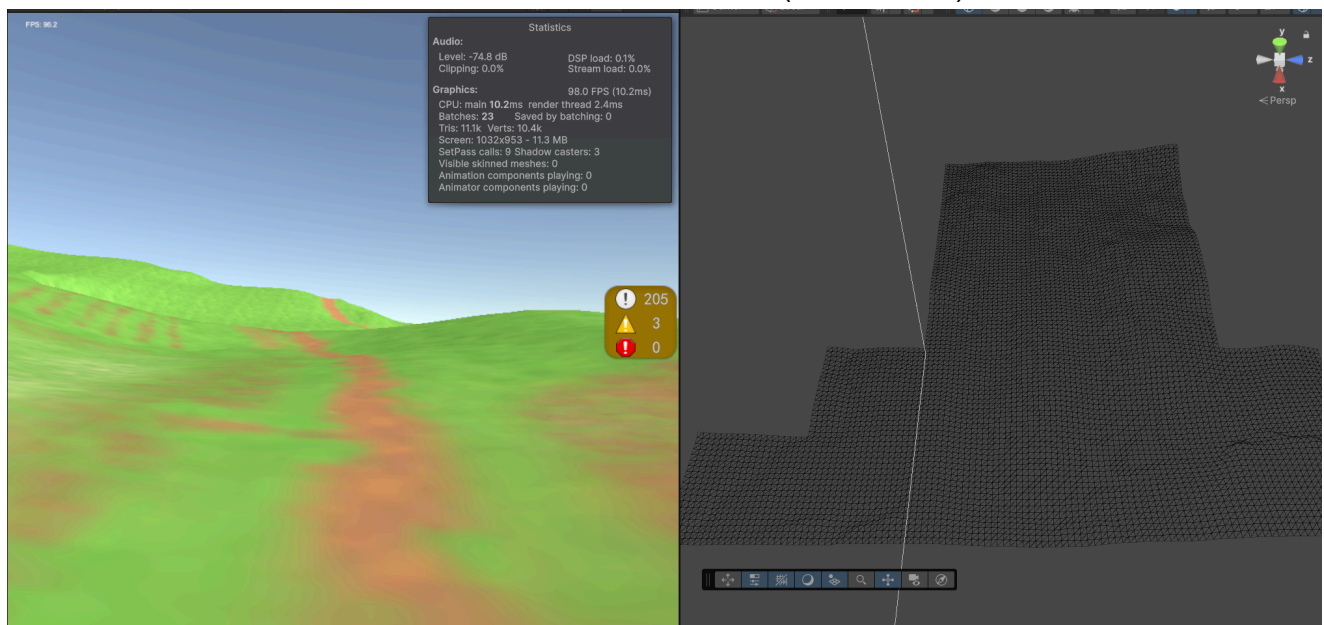


22us，性能消耗非常低（之前地形的消耗是499us）

地形不会根据权重图去融合各个layer，所有layer都已经烘焙到一起了，因此消耗比较低。不过unity的terrain，本身也有一个BaseMap距离，也是同样的原理，这个就没有去测了。

vkCmdWriteTimestamp(Query Pool 168, 3469)	
DrawOpaqueObjects	137.34375
RenderLoop.DrawSRPBatcher	22.8125
RenderLoop.Draw	114.53125
vkCmdWriteTimestamp(Query Pool 168, 3470)	
vkCmdEndRenderPass()	

地形转了网格，只加载可见视野内的块，面数只有9.4k (之前是3.4w)



这里对地形的简化网格和layer烘焙，terrain本身都有

所以能得出结论，terrain本身也在网格和渲染方面有相关优化，可以做到比较低的消耗，只是自己去控制能更加具备自由度。

114us 是草和树的批量渲染，本身消耗很低。(之前是 8ms)

草和树的瓶颈主要在剔除 + 面数

1191-2166	DrawOpaqueObjects	137.34375
1192-1285	RenderLoop.DrawSRPBatcher	22.8125
1287-2164	RenderLoop.Draw	114.53125
1288	vkCmdBindPipeline(Graphics Pipeline 11353)	
1289	vkCmdBindDescriptorSets(0, { Descriptor Set 11482, Descriptor Set 11611 })	
1290	vkCmdBindVertexBuffers(0, { FlowerMeadow1 })	
1291	vkCmdBindIndexBuffer(FlowerMeadow1, VK_INDEX_TYPE_UINT16)	
1292	Indirect sub-command({ 36, 17 })	
1293	vkCmdDrawIndexedIndirect(1) => <36, 17>	0.625
1294	vkCmdBindDescriptorSets(0, { Descriptor Set 11481, Descriptor Set 11611 })	
1295	vkCmdBindVertexBuffers(0, { Grass1 })	
1296	vkCmdBindIndexBuffer(Grass1, VK_INDEX_TYPE_UINT16)	
1297	Indirect sub-command({ 108, 237 })	
1298	vkCmdDrawIndexedIndirect(1) => <108, 237>	0.625
1299	vkCmdBindDescriptorSets(0, { Descriptor Set 11480, Descriptor Set 11611 })	
1300	vkCmdBindVertexBuffers(0, { Reeds })	
1301	vkCmdBindIndexBuffer(Reeds, VK_INDEX_TYPE_UINT16)	
1302	Indirect sub-command({ 45, 0 })	
1303	vkCmdDrawIndexedIndirect(1) => <45, 0>	0.57292
1304	vkCmdBindDescriptorSets(0, { Descriptor Set 11479, Descriptor Set 11611 })	
1305	vkCmdBindVertexBuffers(0, { Wheat })	
1306	vkCmdBindIndexBuffer(Wheat, VK_INDEX_TYPE_UINT16)	
1307	Indirect sub-command({ 18, 0 })	

内存消耗

这些是computer shader 所使用到的buffer， 占182m， 主要是草的渲染模块申请的每个实例， float是32位， 4字节。 double是64位， 8字节

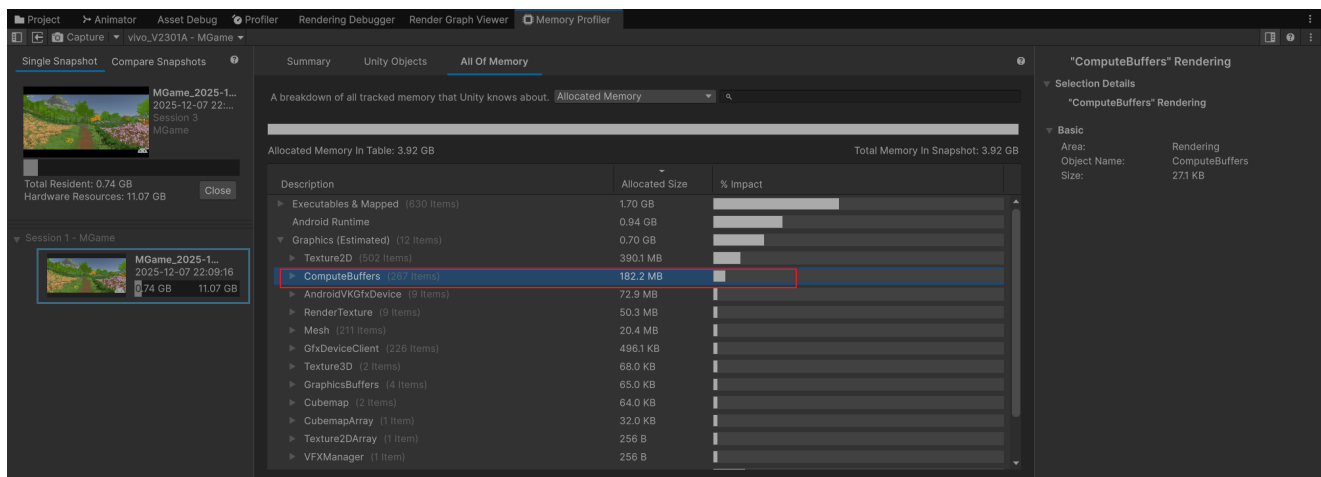
float4x4就是16个float， $16 \times 4 = 64$ 字节

一个float4是 4个float的意思， 也就是 $4 \times 4 = 16$ 字节

因此 $64 + 16 + 16 = 96$

$96 \times 168w = 153m$ 再加上树的， 以及其他计算用到的buffer， 基本对上了

```
struct InstanceData
{
    float4x4 transformMatrix;
    float4 boundsCenter;
    float4 boundsExtents;
};
```



结论

- 1) 帧率从19帧提升到100帧
- 2) 地形转网格，分块，烘焙，流式加载。对比terrain提供的优化，提升有限，但自由度更高。
- 3) gpu instance 实例化生成草，剔除比较耗时2ms，但剔除能减少大量的实例，大大降低面

数

- 4) 剔除的computer shader 需要用到182m的显存，属于空间换时间。
- 5) 最大的关键点在于把同屏面数从190w降低到22w，gpu的瓶颈在于面数太高，有带宽问题。其次dc的减少和面数的降低，让cpu的submit耗时从9ms降低到0.几ms。
- 6) 优化前后的画面对比，右边的树使用的是面数较低的网格，草的密度也低一点。在移动平台，要跑高帧率，就需要控制面数，把细节和法线烘焙在贴图，这里只是用于展示优化效果，使用已有资源来做，是会难看点。

